

Experiment-7

AIM- To interface an SSD1306 display over I2C and present real-time sensor values on-screen.

APPARATUS- STM32 Discovery Kit, STM32 CUBE IDE, STM32 CubeMX, Realterm, LCD display, connecting wires.

PROCEDURE

1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.
2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L475VGT6.
3. **Clock Selection:** Go to RCC and configure clock as “By pass Clock Source” as shown below:

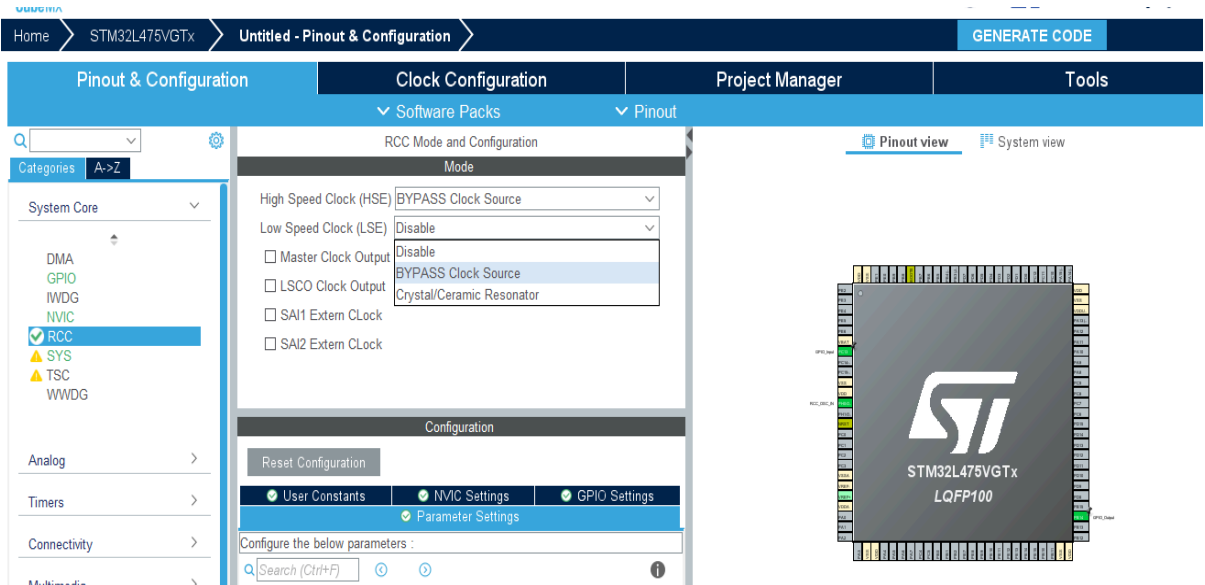


Fig-7.1 – Clock Selection

4. As per user manual of board ST LINK is internally connected to PB6(Tx) and PB7(Rx) of USART1.

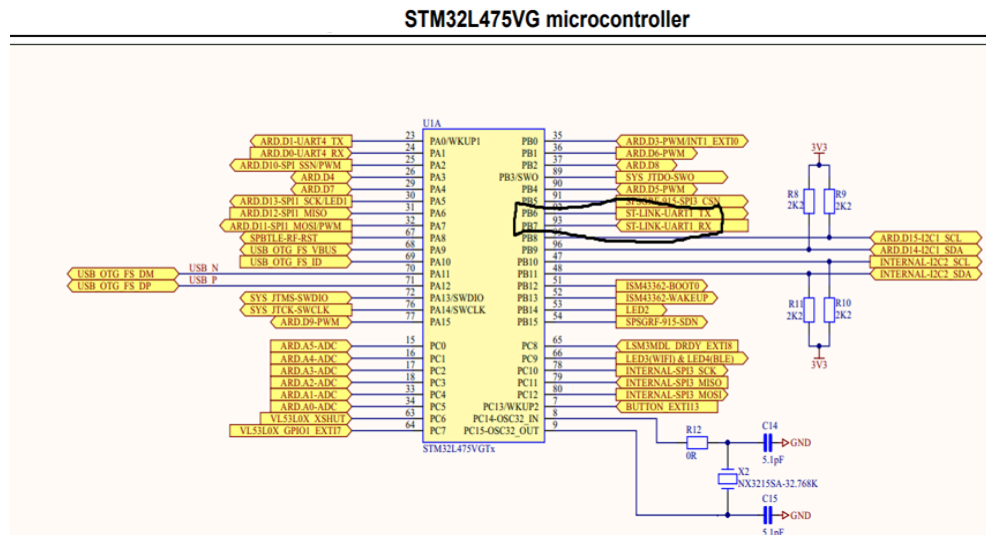


Fig-7.2 – STM32 Board Pin Configuration

- Under “connectivity” click on “USART1” and make PB6 as USART1_Tx and PB7 as USART2_Rx as shown below:

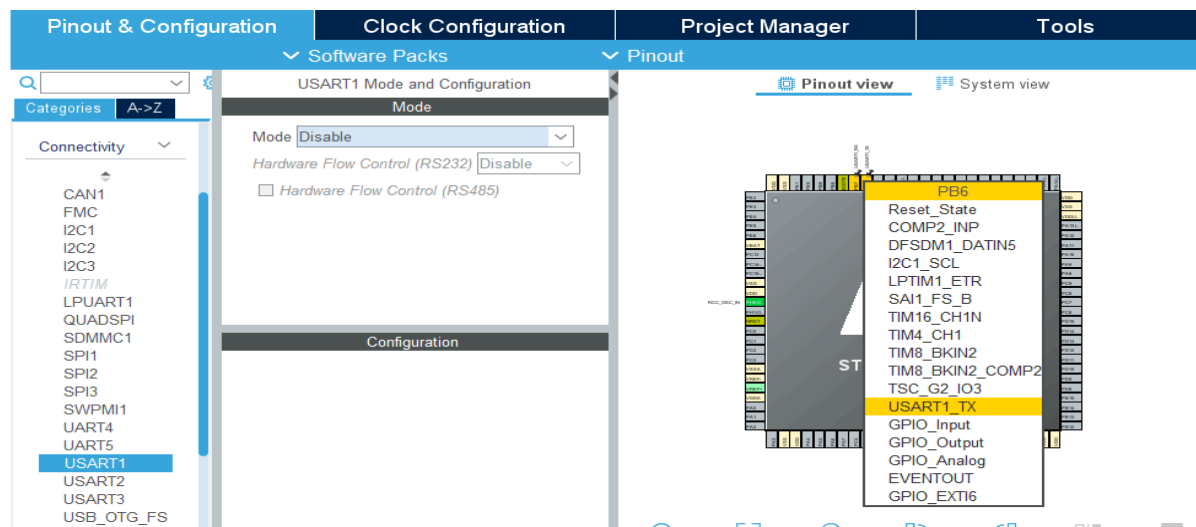


Fig-7.3 – STM32 Board Pinout and Configuration

- Select USART1 and click on “ Asynchronous” in “Mode” as shown below :

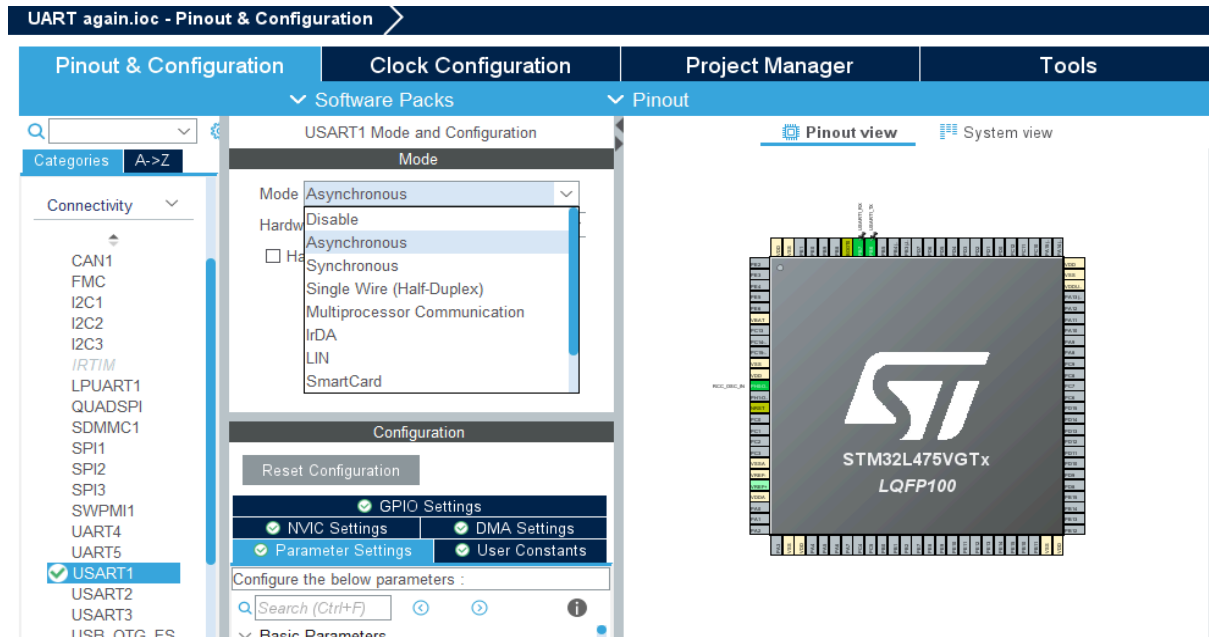


Fig-7.4 – Selection of Asynchronous mode

7. Go to “connectivity” and also select I2C1 in mode.
8. Click on PB8 and select I2C1_SCL to make PB8 pin as clock source pin as shown below:

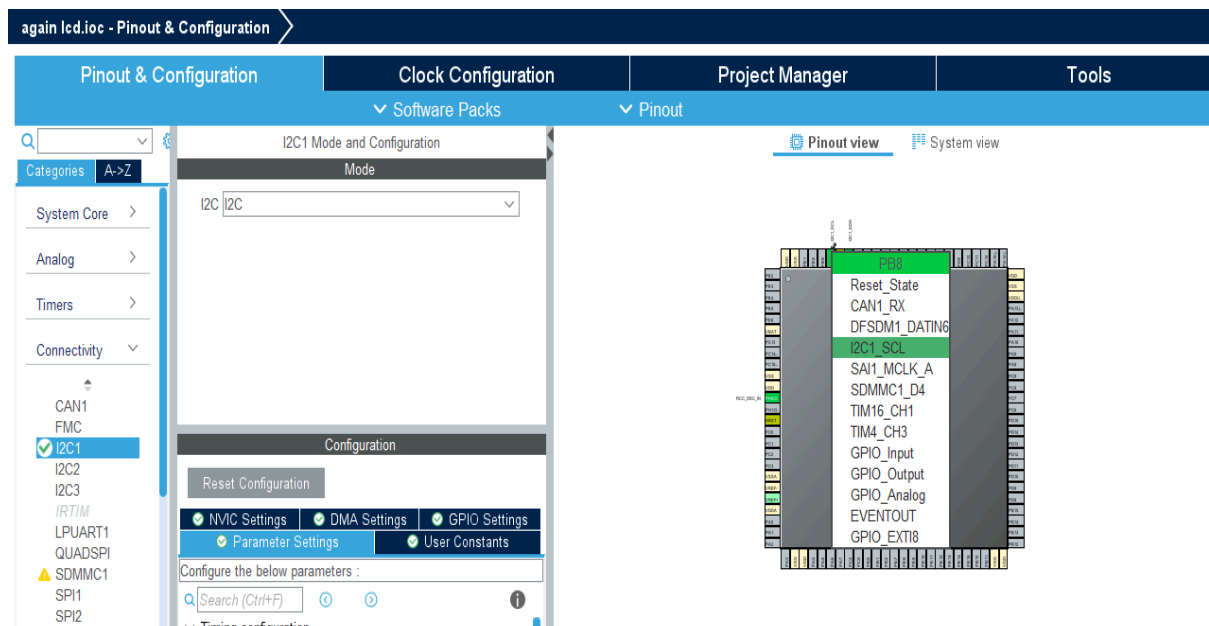


Fig-7.5 – Settings of Basic Parameters

9. **Clock Configuration:** Configure the clock as shown below to get a maximum frequency of 80MHz.

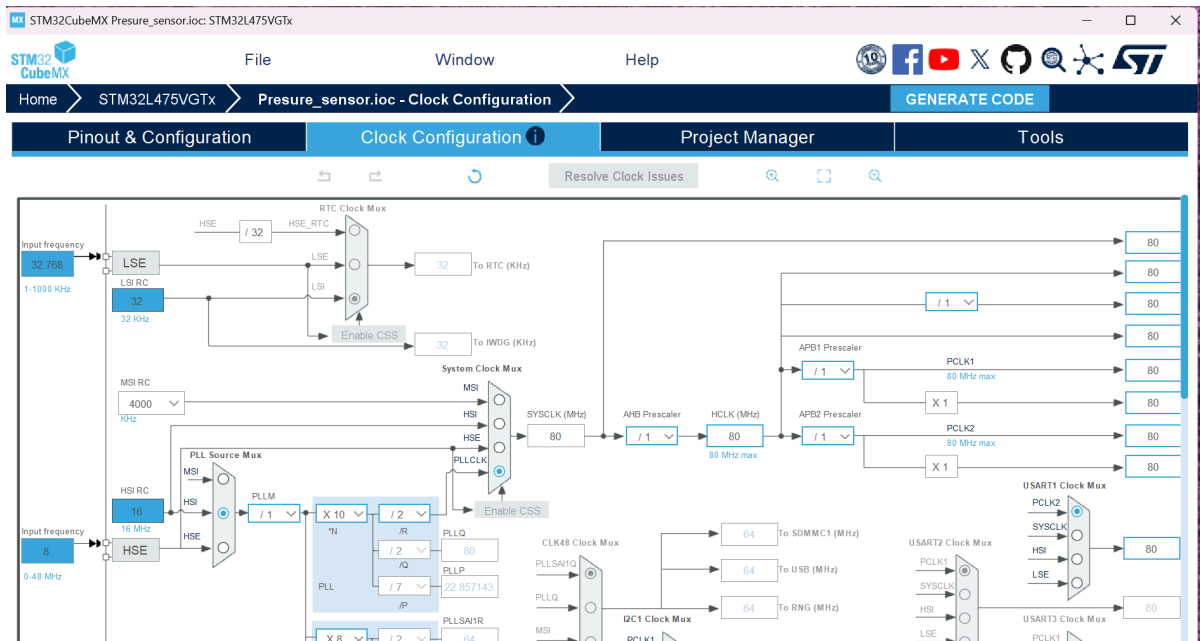


Fig-7.6 – Clock Configuration

10. Write Project name and select CubeIDE.
11. Click on generate code.
12. Click on the open project.
13. Click ‘OK’ on the generated popup showing “successfully imported the project on the work space” and the project will be successfully added to the cube IDE.
14. Copy BSP drivers to your project.
15. The last version of the STM32CubeL4 package is downloaded by default in the STM32CubeMX repository
(C:\Users\user_name\STM32Cube\Repository\STM32Cube_FW_L4_Vx.xx.x\Drivers).

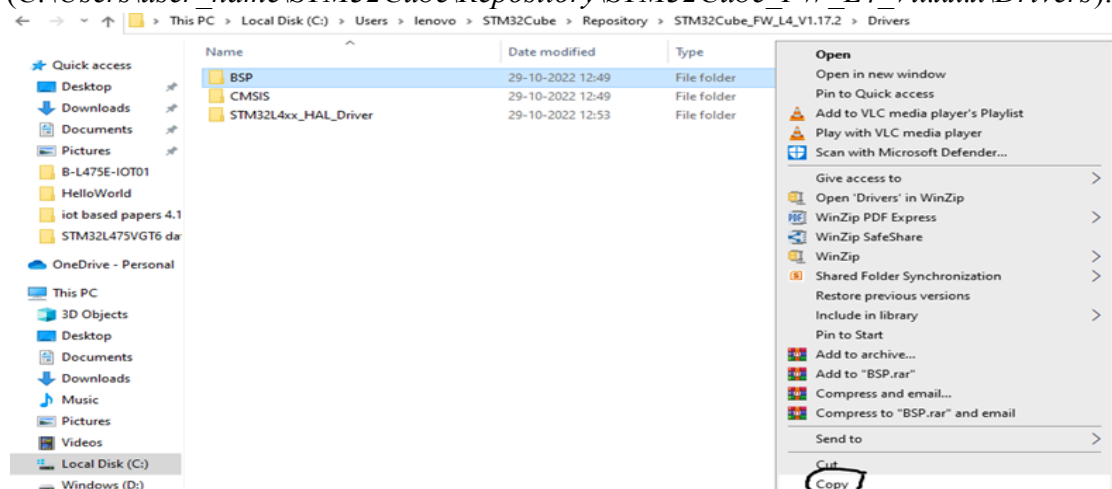


Fig-7.7 – BSP drivers' location

16. Paste the copied BSP drivers in the drive folder as shown below:

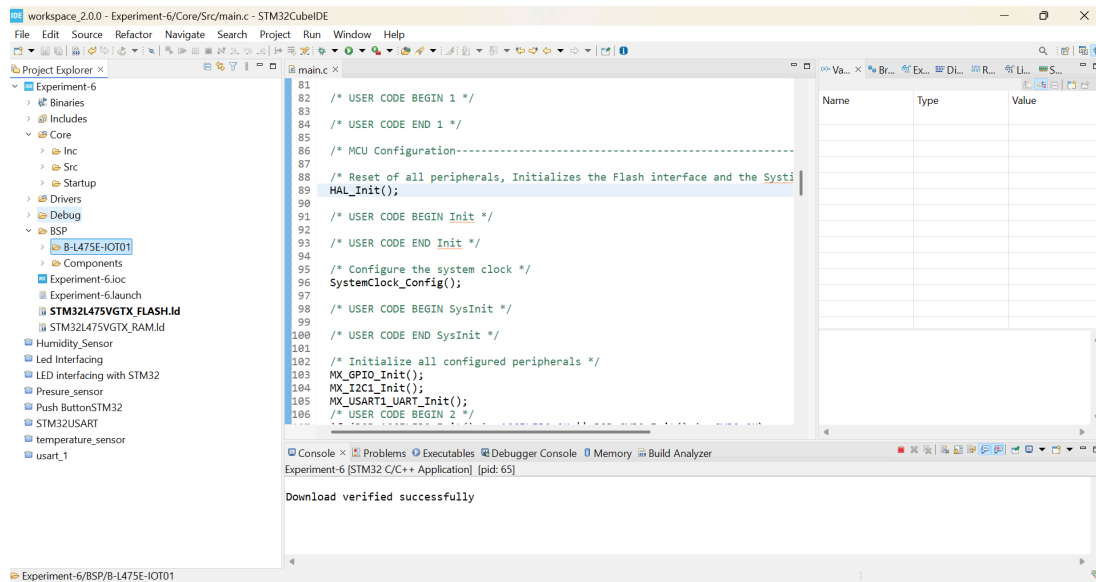


Fig-7.8 – Paste BSP drivers in project folder

17. Delete all folders in BSP, except B-L475E-IOT01 and the component as shown:

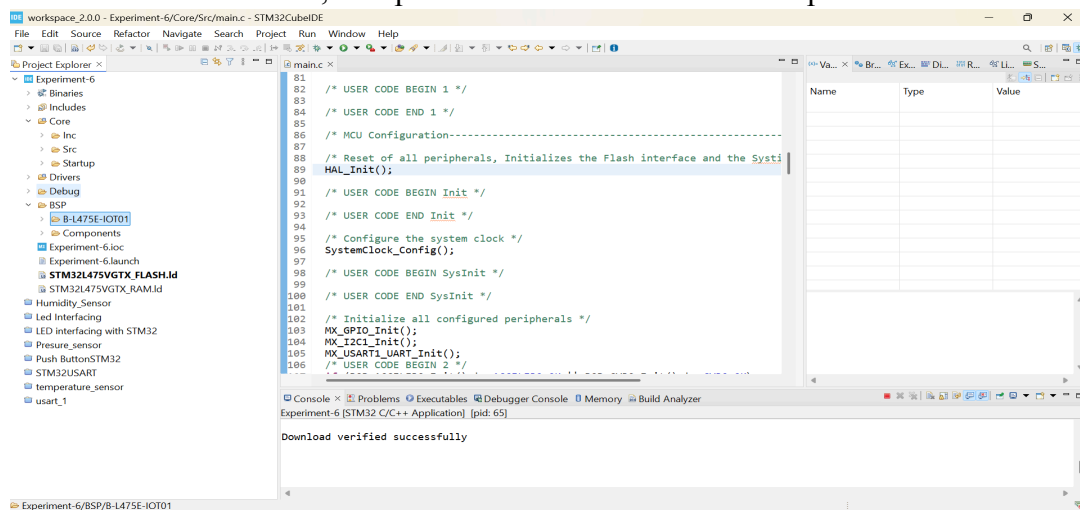


Fig-7.9 – B-L475E-IOT01 and component folder in BSP drivers

18. Keep only these folders in B-L475E-IOT01 and the component as shown in figure:

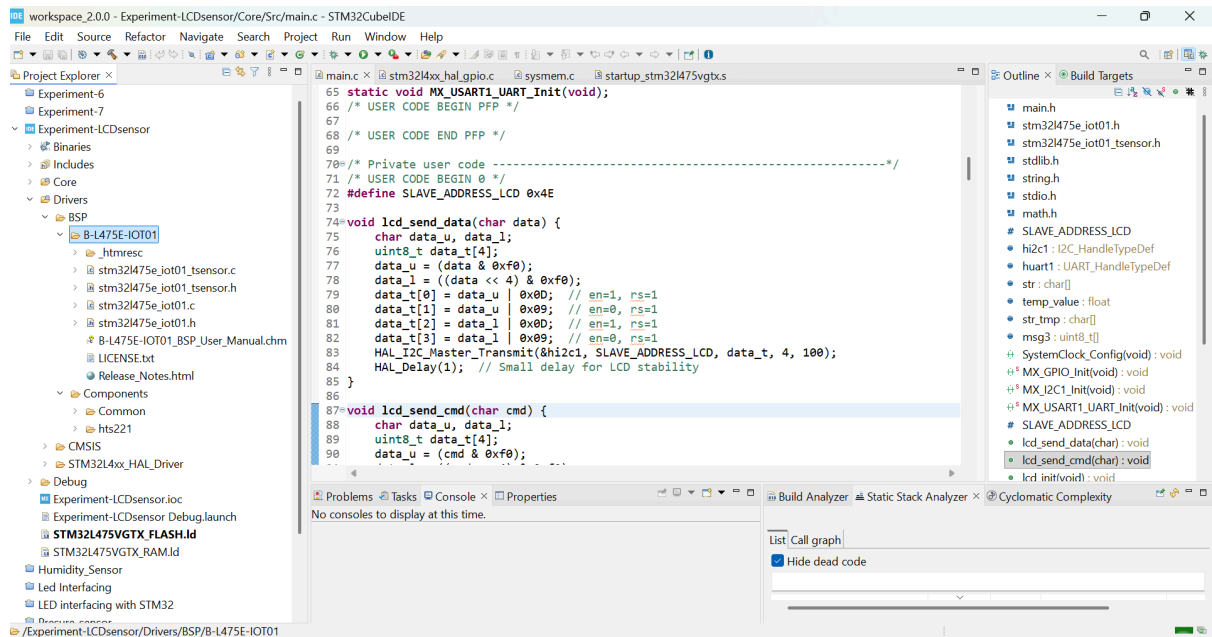


Fig-7.10 – Folders in B-L475E-IOT01 and Component

19. Update include paths: From Project menu or File menu, go to Properties > C/C++ Build > Settings > Tool Settings > MCU GCC Compiler > Include paths and click “add” icon.png to include the new paths as shown below:

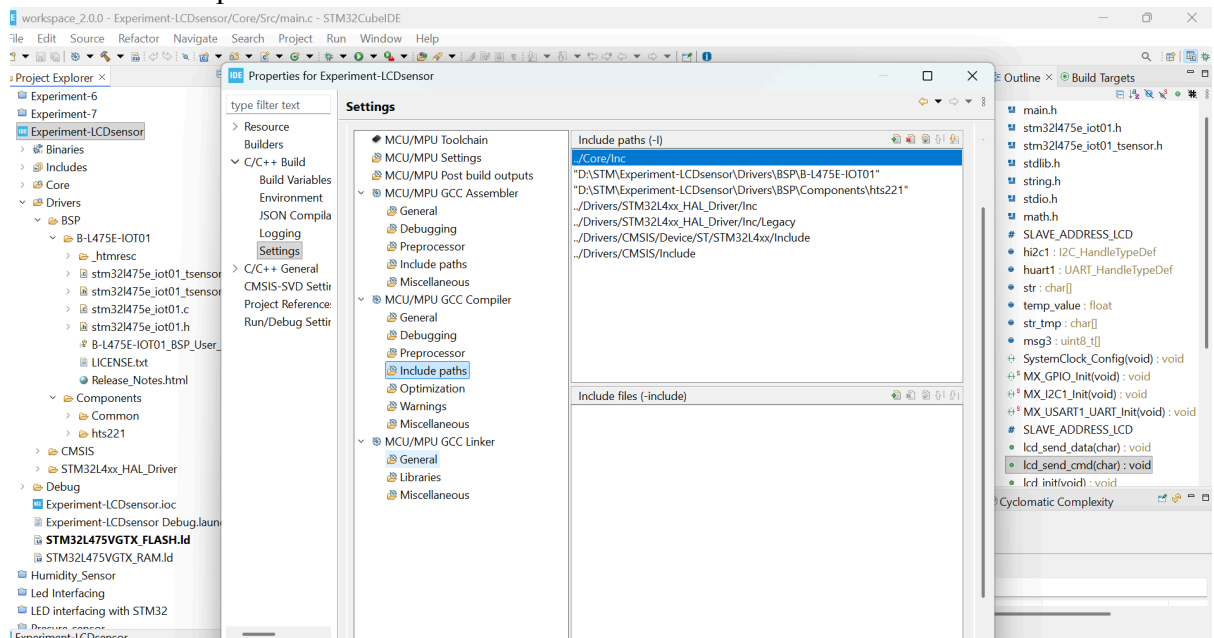


Fig-7.11 – Path settings

20. Add ../Drivers/BSP/B-L475E-IOT01 and ../Drivers/BSP/Components/lsm6dsl paths as addresses given below :
C:\Users\lenovo\STM32Cube\Repository\STM32Cube_FW_L4_V1.17.2\Drivers\BSP\B-L475E-IOT01“ and

C:\Users\lenovo\STM32Cube\Repository\STM32Cube_FW_L4_V1.17.2\Drivers\BSP\Components\hts221

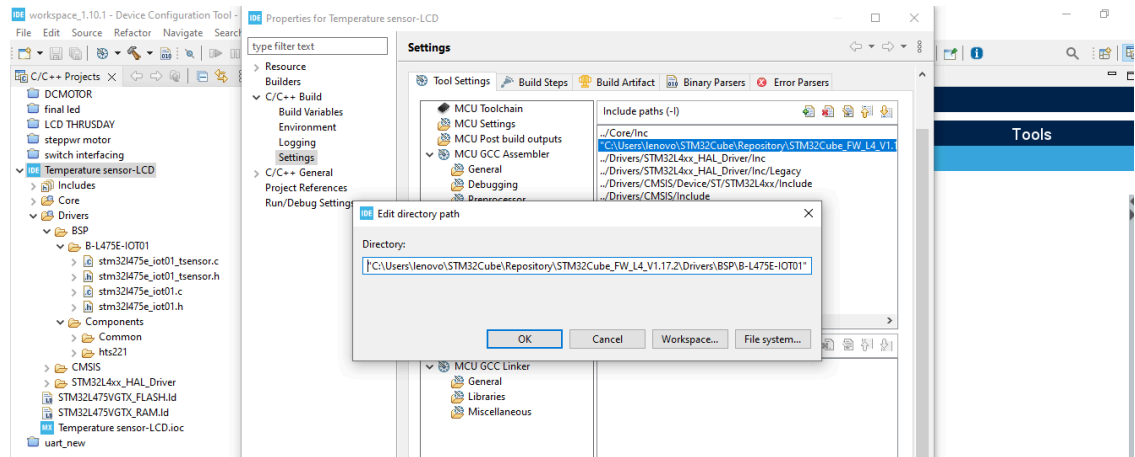


Fig-7.12 – Folders Path settings

21. After adding both path click “ apply and close” as shown below:

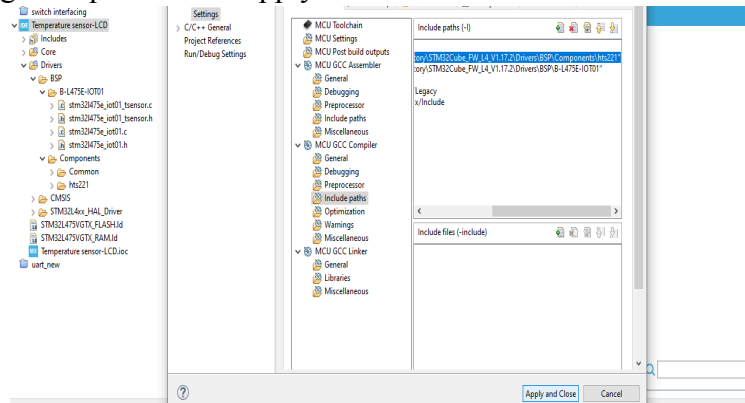


Fig-7.13 – Apply and close setting

22. Go to properties of project>go to settings>go to MCU/MPU settings>Click to float as >apply and close as shown in figure:

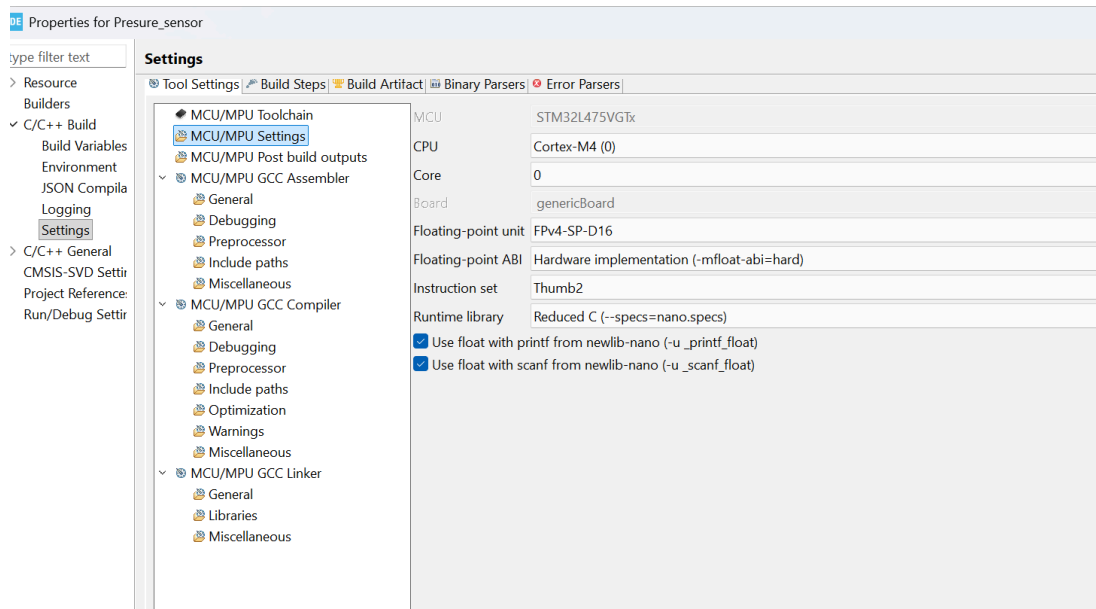


Fig-7.14 – MCU/MPU setting

23. Then “Build” and go to main.c file.

24. Include the header files in main.c files.

```
/* Includes */
#include "stm32l475e_iot01.h"
#include "stm32l475e_iot01_tsensor.h"
#include "stdlib.h"
#include "string.h"
#include "stdio.h"
#include <math.h>
#define SLAVE_ADDRESS_LCD 0x4E
```

25. Write the following instructions:

```
/* USER CODE BEGIN PV */
char str[40];
float temp_value = 0; // Measured temperature value
char str_tmp[100] = ""; // Formatted message to display the temperature value
uint8_t msg3[] = "====> Temperature sensor HTS221 initialized \r\n ";
/* USER CODE END PV */
```

26. Write the following instructions as given:

```
/* USER CODE BEGIN 0 */
#define SLAVE_ADDRESS_LCD 0x4E

void lcd_send_data(char data) {
    char data_u, data_l;
    uint8_t data_t[4];
    data_u = (data & 0xf0);
    data_l = ((data << 4) & 0xf0);
```



```
data_t[0] = data_u | 0x0D; // en=1, rs=1
data_t[1] = data_u | 0x09; // en=0, rs=1
data_t[2] = data_l | 0x0D; // en=1, rs=1
data_t[3] = data_l | 0x09; // en=0, rs=1
HAL_I2C_Master_Transmit(&hi2c1, SLAVE_ADDRESS_LCD, data_t, 4, 100);
HAL_Delay(1); // Small delay for LCD stability
}

void lcd_send_cmd(char cmd) {
    char data_u, data_l;
    uint8_t data_t[4];
    data_u = (cmd & 0xf0);
    data_l = ((cmd << 4) & 0xf0);
    data_t[0] = data_u | 0x0C; // en=1, rs=0
    data_t[1] = data_u | 0x08; // en=0, rs=0
    data_t[2] = data_l | 0x0C; // en=1, rs=0
    data_t[3] = data_l | 0x08; // en=0, rs=0
    HAL_I2C_Master_Transmit(&hi2c1, SLAVE_ADDRESS_LCD, data_t, 4, 100);
    HAL_Delay(2); // Command needs more delay
}

void lcd_init(void) {
    HAL_Delay(50);
    lcd_send_cmd(0x30);
    HAL_Delay(5);
    lcd_send_cmd(0x30);
    HAL_Delay(5);
    lcd_send_cmd(0x30);
    HAL_Delay(5);
    lcd_send_cmd(0x20); // 4-bit mode
    HAL_Delay(5);

    lcd_send_cmd(0x28); // 4-bit, 2 lines
    lcd_send_cmd(0x08); // Display off
    lcd_send_cmd(0x01); // Clear
    HAL_Delay(2);
    lcd_send_cmd(0x06); // Entry mode
    lcd_send_cmd(0x0C); // Display on, no cursor
}

void lcd_clear(void) {
    lcd_send_cmd(0x01);
    HAL_Delay(2);
}

void lcd_set_cursor(uint8_t col, uint8_t row) {
    uint8_t address;
    if (row == 0) {
        address = 0x80 + col;
    }
}
```

```
    } else {  
        address = 0xC0 + col;  
    }  
    lcd_send_cmd(address);  
}  
  
void lcd_send_string(char *str) {  
    while (*str) {  
        lcd_send_data(*str++);  
    }  
}
```

```
/* USER CODE END 0 */
```

27. Write the following instructions as given:

```
/* USER CODE BEGIN 2 */  
    BSP_TSENSOR_Init();  
    HAL_UART_Transmit(&huart1, msg3, sizeof(msg3), 1000);  
    // Initialize LCD once here  
    lcd_init();  
    HAL_Delay(100);  
/* USER CODE END 2 */
```

28. Write the following instructions in while (1) loop under user code begin 3.

```
    temp_value = BSP_TSENSOR_ReadTemp();  
    snprintf(str_tmp, sizeof(str_tmp), "Temp: %.2f C\r\n", temp_value);  
    HAL_UART_Transmit(&huart1, (uint8_t*)str_tmp, strlen(str_tmp), 1000);  
    if (!lcd_initialized)  
    {  
        lcd_init();  
        lcd_initialized = 1;  
    }  
  
    lcd_clear();  
    lcd_set_cursor(0, 0); // <- This uses the function  
    lcd_send_string(str_tmp);  
  
    HAL_Delay(1000);
```

29. Now connect the board and click on “Build”, “debug”.

30. Click on “Switch” on popup and click on “Resume”, the program will start execute.

31. Now, open the console “Real term” and set the baud rate as used in program (in this program it is 115200), select port (as per use) and click on “change” on the real term console. Also check the value of temperature on LCD.

OBSERVATION: The output values of temperature will display on real term as well as on LCD as shown below:

Reflection Summary

Reflect on today's experiment in which you interfaced an LCD over I2C to display real-time temperature sensor values using the STM32L475E-IOT01 board and STM32CubeIDE. Answer the following sub-questions in a structured manner:

a) Explain the main aim of this experiment in your own words and briefly describe how the combination of I2C, UART and the temperature sensor helped to achieve this aim.

The main aim of this experiment was to interface an LCD display using the I2C protocol with the STM32L475E-IOT01 board and display real-time temperature sensor values.

In this experiment three main components were used:

- HTS221 Temperature Sensor – measures the temperature from the board.
- I2C Communication – used to send data from the microcontroller to the LCD display.
- UART Communication – used to send the temperature values to the RealTerm terminal for monitoring.

The sensor reads the temperature and sends the value to the microcontroller. The STM32 then formats this data and sends it simultaneously to the LCD through I2C and to the PC terminal through UART, allowing real-time monitoring.

b) Describe the key configuration steps you performed in STM32CubeMX for USART1 and I2C1, and explain why these steps were essential for communicating with RealTerm and the LCD.

USART1 Configuration

1. USART1 was enabled in Asynchronous mode.
2. PB6 was configured as USART1_TX and PB7 as USART1_RX.
3. Baud rate was set to 115200 for serial communication with RealTerm.

Purpose:

This allowed the STM32 board to transmit temperature data to the computer terminal for debugging and monitoring.

I2C1 Configuration

1. I2C1 peripheral was enabled.
2. PB8 configured as I2C1_SCL (clock line).
3. PB9 configured as I2C1_SDA (data line).

Purpose:

These pins allow the STM32 to communicate with the LCD module using the I2C protocol, which requires only two wires for communication.

These configurations were essential because UART handles PC communication while I2C handles LCD communication.

c) Reflect on the process of adding BSP drivers and updating include paths for the STM32L475E-IOT01 board. What specific difficulties did you encounter, and how did you overcome them?

To use onboard sensors like HTS221, the Board Support Package (BSP) drivers were required.

Steps performed

- BSP drivers were copied from the STM32Cube repository.
- Only required folders such as B-L475E-IOT01 and Components were kept.
- Include paths were added in
Project → Properties → C/C++ Build → Include Paths.

Difficulties faced

- Incorrect driver paths initially caused compilation errors.
- Missing header files prevented sensor initialization

Solution

The correct directories for BSP and sensor components were manually added to the include paths. After updating these paths, the project compiled successfully.

d) Discuss how the `lcd_init`, `lcd_send_cmd` and `lcd_send_data` functions work together to correctly initialize and drive the LCD in 4-bit mode. What important concepts about low-level peripheral control did you learn from writing or using these functions?

1. `lcd_init()`

This function initializes the LCD by sending a sequence of commands to configure it in 4-bit mode, clear the display, and enable the cursor.

2. `lcd_send_cmd()`

This function sends control commands to the LCD such as:

- Clear display
- Set cursor position

- Turn display on/off

3. lcd_send_data()

This function sends actual characters or data to the LCD for display.

How they work together

1. lcd_init() prepares the LCD.
2. lcd_send_cmd() configures display settings.
3. lcd_send_data() writes characters to the screen.

Key concepts learned

- Low-level hardware communication
- Bit manipulation
- Sending commands using the I2C protocol
- LCD communication in 4-bit model

e) Explain how the HTS221 temperature sensor was initialized and used inside the while(1) loop. How was the temperature value formatted and sent simultaneously to both the UART terminal and the LCD, and what does this teach you about data representation and display in embedded systems?

The HTS221 sensor was initialized using:

```
BSP_TSENSOR_Init();
```

Inside the while(1) loop, the temperature was read using:

```
temp_value = BSP_TSENSOR_ReadTemp();
```

Data formatting

The temperature value was converted into a readable string using:

```
snprintf(str_tmp, sizeof(str_tmp), "Temp: %.2f C\r\n", temp_value);
```

Data display

- Sent to UART terminal using HAL_UART_Transmit()
- Sent to LCD display using lcd_send_string()

This demonstrates how embedded systems convert sensor data into human-readable information and display it on multiple interfaces simultaneously

f) Compare the roles of UART and I2C in this experiment in terms of their purpose, configuration effort and ease of debugging. Which interface did you personally find easier to understand and use, and why? Support your answer with specific examples from your lab work.

In this experiment, UART and I2C were used for different communication purposes. UART was used for communication between the STM32 microcontroller and the computer (RealTerm terminal), while I2C was used for communication between the STM32 and the LCD display module.

Comparison Table:

Parameter	UART	I2C
Purpose in this experiment	Used to send temperature data from STM32 to the PC terminal (RealTerm)	Used to send commands and data from STM32 to the LCD display
Communication type	Asynchronous Serial communication	Synchronous serial communication
Number of wires	2 wires (TX and RX)	2 wires (SCL – clock, SDA – data)
Configuration in CubeMX	USART1 enabled in asynchronous mode with PB6 as TX and PB7 as RX	I2C1 enabled with PB8 as SCL and PB9 as SDA
Speed control	Determined by baud rate	Determined by clock frequency

Personal observation

UART was easier to understand because:

- Data can be viewed directly on the PC terminal.
- Errors can be easily identified.

I2C required additional understanding of addressing and command formatting, which made debugging slightly more difficult.

g) Suppose you want to extend this system to show additional sensor values or system status messages on the LCD. What changes (in code structure, driver usage or display logic) would you plan, and how does this experiment prepare you for that extension?

If the system needs to display more sensor values or system status, the following changes can be made:

Code changes

- Create additional sensor reading functions.
- Modify the display logic to handle multiple values.

Display improvements

- Use multiple LCD rows to show different parameters.
- Implement scrolling messages.

Driver usage

- Add BSP drivers for other sensors (humidity, pressure, etc.).

Example extensions

- Display temperature and humidity together
- Show system status messages
- Display sensor alerts

This experiment prepares us for these extensions by teaching sensor interfacing, communication protocols, and real-time display techniques in embedded systems.